



Task 3 - paradox.c (Mortimer)

Anthony Mortimer posted Oct 4, 2026 10:53 AM  [Subscribe](#)



These were my timings for each thread count (median of three runs):

Threads	A	B
1	0.217	0.211
2	0.222	0.216
4	0.248	0.219
8	0.369	0.230
12	0.524	0.233

Sums:

Threads	Forward	Reverse	Difference (rounded)
1	1e+19	1e+19	1e+19
2	0	0	0
4	199.999999679081840	0	200
8	199.999999679081840	0	200
12	199.999999679081840	0	200

Since each thread gets an even division of work, I expected the run time to not increase as threads were added. However, run A slowed as threads were added, but run B stayed consistent. The sums also matched with 1 and 2 threads, but had a difference of 200 from 4+ threads.

I've never personally worked directly with memory caching techniques, so some of these terms are new for me. The only major difference I see between run_a and run_b is how sums are stored per thread. run_a uses a plain double array, while run_b uses a struct that takes into account cache lines. My guess is that run_b takes advantage of caching techniques, while



run_a does not. That may explain the degradation in performance as threads are added for run_a, while run_b stays consistent.

I saw the local double that was commented out in the program and used that when adding to the sum. My times for run_a had better performance than before, but I honestly can't explain why. I imagine it's something related to a cache hit, or not needing to even leave the CPU to access the local double?

Threads	A	B
4	0.0946	0.0946
8	0.1593	0.1009
12	0.1590	0.0953

Reply to Thread

Filter by:

All Posts

 | Clear filters

Show:

Threaded

There are no replies in this thread

Reply to Thread

